

SVEUČILIŠTE JURJA DOBRILE U PULI
FAKULTET INFORMATIKE

Tin Pritišanac

Analiza tržišta automobila 1970.-2024.

SEMINARSKI RAD

Pula, rujan, 2025. godine

SVEUČILIŠTE JURJA DOBRILE U PULI
FAKULTET INFORMATIKE

Tin Pritišanac

Analiza tržišta automobila 1970.-2024.

SEMINARSKI RAD

JMBAG: 0171256219, izvanredni student
Studijski smjer: Informatika
Kolegij: Skladišta i rudarenje podataka
Mentor: doc.dr.sc. Goran Oreški

Pula, rujan, 2025. godine



IZJAVA O AKADEMSKOJ ČESTITOSTI

Ja, dolje potpisani Pritišanac Tin, ovime izjavljujem da je ovaj seminarski rad rezultat isključivo mogega vlastitog rada, da se temelji na mojim istraživanjima te da se oslanja na objavljenu literaturu kao što to pokazuju korištene bilješke i bibliografija. Izjavljujem da niti jedan dio Završnog rada nije napisan na nedozvoljen način, odnosno da je prepisan iz kojega necitiranog rada, te da ikoji dio rada krši bilo čija autorska prava. Izjavljujem, također, da nijedan dio rada nije iskorišten za koji drugi rad pri bilo kojoj drugoj visokoškolskoj, znanstvenoj ili radnoj ustanovi.

Student

U Puli, rujan, 2025. godine

Sadržaj

1	Uvod	1
2	Projektni zadatak	2
2.1	Definicija problema i motivacija	2
2.2	Cilj i opis projektnog zadatka	2
2.3	Opseg i ograničenja projekta	2
2.4	Očekivani rezultati i doprinosi	3
2.5	Metodologija rada	3
3	Odabir i analiza skupa podataka	5
3.1	Izvor i kriteriji odabira podataka	5
3.2	Eksploratorna analiza podataka	5
3.2.1	Osnovna struktura podataka	5
3.2.2	Ključni uvidi iz eksploratorne analize	6
3.3	Priprema i obrada podataka	7
3.3.1	Čišćenje i standardizacija podataka	7
3.3.2	Proširenje skupa podataka hijerarhijskim atributima	8
3.3.3	Podjela skupa podataka za simulaciju različitih izvora	9
3.4	Rezultati pripreme podataka	10
4	Relacijski model podataka	11
4.1	Analiza entiteta i atributa	11
4.2	Konceptualni model podataka	11
4.3	Implementacija relacijskog modela	12
4.4	Automatizacija kreiranja i popunjavanja baze	13
4.5	Validacija podataka i testiranje	14
5	Dimenzijski model podataka	16
5.1	Konceptualni pristup dimenzijskom modeliranju	16
5.2	Arhitektura zvjezdastog modela	16
5.3	Tablica činjenica - Fact_Car_Sales	18
5.4	Dimenzijske tablice i SCD strategije	18
5.5	Prednosti zvjezdastog modela	20
5.6	Implementacijski detalji	20
6	ETL proces	22
6.1	Arhitektura ETL pipelinea	22
6.2	Extract faza - izvlačenje podataka	23
6.3	Transform faza - transformacija podataka	23
6.3.1	Transformacija dimenzijskih tablica	24
6.3.2	Transformacija tablice činjenica	25
6.4	Load faza - učitavanje u skladište podataka	26
6.5	Orkestracija i monitoring	28
6.6	Prednosti implementiranog ETL rješenja	28

7	OLAP analiza	30
7.1	Definiranje prikaza podataka	30
7.2	Vizualizacija podataka u Tableau	30
7.2.1	Graf 1 TODO	30
8	Zaključak	31
	Literatura	32
	Popis slika	34
	Popis tablica	35

1 Uvod

U današnjem digitalno vođenom poslovnom okruženju, sposobnost efikasnog prikupljanja, obrade i analize velikih količina podataka postala je ključni čimbenik uspjeha za organizacije u gotovo svim industrijskim granama. Automobilska industrija, kao jedna od najkompleksnijih i najkonkurentnijih grana gospodarstva, posebno se oslanja na napredne tehnologije skladištenja i analize podataka kako bi razumjela tržišne trendove, potrebe kupaca i operacijske učinkovitosti [1].

Skladišta podataka (eng. *data warehouses*) predstavljaju temelj modernih sustava za podršku odlučivanju, omogućujući integraciju različitih izvora podataka u jedinstvenu, koherentnu strukturu optimiziranu za analitičke potrebe [2]. U kontekstu automobilske industrije, ovakvi sustavi omogućuju analizu složenih odnosa između cijena vozila, karakteristika proizvođača, tržišnih segmenata i regionalnih specifičnosti.

Ovaj rad predstavlja sveobuhvatan pristup razvoju sustava za skladištenje i rudarenje podataka primjenjenog na analizu automobilskog tržišta. Kroz razvoj kompletnog ETL (Extract, Transform, Load) procesa, projekt demonstrira transformaciju sirovih podataka o automobilima u strukturirani dimenzijski model prilagođen OLAP (Online Analytical Processing) analizama. Korištenjem skupa podataka koji sadrži preko 97.000 zapisa o vozilima različitih proizvođača, modela i karakteristika, razvijen je sustav koji omogućuje dubinsku analizu tržišnih trendova i poslovnih uvida.

Glavni cilj ovog projekta je ilustracija praktične primjene teorijskih koncepata skladišta podataka kroz razvoj funkcionalnog sustava koji može poslužiti organizacijama poput autoklubova, analitičkih kuća ili samim proizvođačima automobila u donošenju informiranih poslovnih odluka [3]. Projekt obuhvaća sve ključne faze razvoja sustava - od eksploratorne analize početnih podataka, preko dizajna relacijskog i dimenzijskog modela, do implementacije ETL procesa i prijedloga OLAP analiza.

Struktura rada prati logični tijek razvoja sustava, počevši od analize i pripreme početnog skupa podataka, preko stvaranja normaliziranog relacijskog modela, do konačne implementacije zvjezdastog modela optimiziranog za analitičke potrebe. Svaki korak popraćen je detaljnim objašnjenjima projektnih odluka i praktičnih implementacijskih izazova, čineći ovaj rad korisnim resursom za razumijevanje kompleksnosti razvoja realnih sustava za skladištenje podataka.

Kroz ovaj projekt, nastoji se pokazati kako tehnologije poput Apache Spark-a, MySQL-a i Tableau-a mogu biti integrirane u koherentan sustav koji omogućuje ne samo tehnički ispravan rad, već i stvaranje dodane vrijednosti kroz kvalitetne poslovne uvide [4].

2 Projektni zadatak

2.1 Definicija problema i motivacija

Automobilska industrija je jedna od najkompleksnijih gospodarskih grana koja generira ogromne količine podataka - od osnovnih karakteristika vozila, preko cijena i tržišnih trendova, do regionalnih specifičnosti i preferencija kupaca. Organizacije koje se bave analizom automobilskeg tržišta, poput autoklubova, analitičkih kuća ili samih proizvođača, suočavaju se s izazovom pretvaranja ovih velikih količina podataka u korisne poslovne uvide.

Tradicionalni pristup analize podataka korištenjem jednostavnih baza podataka i osnovnih alata za izvještavanje često se pokazuje neadekvatnim za složene analitičke potrebe. Potreba za dubinskim analizama tržišnih trendova, usporednim analizama proizvođača, segmentacijom kupaca i predviđanjem buduće dinamike tržišta zahtijeva sofisticiraniji pristup [1].

Upravo tu se prepoznaje potreba za razvojem naprednog sustava skladištenja i analize podataka koji će omogućiti organizacijama da iz sirovih podataka izvuku maksimalnu vrijednost te donose informirane strateške odluke.

2.2 Cilj i opis projektnog zadatka

Glavni cilj ovog projekta je razvoj potpunog sustava za skladištenje i rudarenje podataka prilagođenog analizi automobilskeg tržišta. Projekt obuhvaća kompletan tijek rada - od početnih sirovih podataka do konačnih analitičkih izvještaja koji mogu koristiti stvarnim organizacijama u njihovom poslovanju.

Konkretno, projekt ima za cilj:

- **Dizajnirati i implementirati relacijski model podataka** koji odgovara strukturi realnih podataka o automobilima, uključujući sve važne entitete i njihove međusobne odnose
- **Razviti dimenzijski model (star schema)** optimiziran za OLAP analize, koji omogućuje efikasno izvršavanje složenih analitičkih upita
- **Implementirati ETL proces** koji transformira podatke iz relacijske strukture u dimenzijski model koristeći suvremene tehnologije poput Apache Spark-a
- **Kreirati sustav za OLAP analize** koji demonstrira praktičnu primjenu različitih analitičkih operacija (slice, dice, drill-down, roll-up, pivot)
- **Pokazati poslovnu vrijednost** kroz konkretne scenarije korištenja koji ilustriraju kako ovakav sustav može koristiti organizacijama u donošenju poslovnih odluka

2.3 Opseg i ograničenja projekta

Projekt se fokusira na analizu podataka o automobilima prodavanima na fiktivnom tržištu, koristeći skup od preko 97.000 zapisa koji obuhvaća vozila različitih

proizvođača, modela i karakteristika. Ovaj skup podataka pruža reprezentativan uzorak koji omogućuje demonstraciju svih ključnih koncepata skladišta podataka.

Vremenski okvir podataka pokriva razdoblje od 1970. do 2024. godine, s naglaskom na zadnja dva desetljeća, što omogućuje analizu dugoročnih trendova i promjena na tržištu. Podaci uključuju ključne attribute poput cijene, godine proizvodnje, kilometraže, vrste goriva, veličine motora i drugih karakteristika relevantnih za tržišnu analizu.

Projekt se ograničava na demonstraciju tehničkih mogućnosti i metodologija, a ne pretendira na potpunu komercijalnu implementaciju. Fokus je na edukacijskim aspektima i ilustraciji najboljih praksi u razvoju sustava za skladištenje podataka.

2.4 Očekivani rezultati i doprinosi

Na završetku projekta očekuje se sljedeće:

1. **Funkcionalno skladište podataka** s implementiranim relacijskim i dimenzijskim modelom podataka
2. **Potpuno funkcionalan ETL proces** koji automatizira transformaciju podataka između različitih modela
3. **Demonstracija OLAP mogućnosti** kroz konkretne analitičke scenarije i upite
4. **Dokumentacija procesa** koja može služiti kao vodič za buduće slične projekte
5. **Praktični uvidi u tržište automobila** koji ilustriraju poslovnu vrijednost analitičkih sustava

Ovaj projekt predstavlja praktičnu demonstraciju kako teorijski koncepti skladišta podataka i business intelligence mogu biti primijenjeni u realnom kontekstu, pružajući studentima i praktičarima vrijedan uvid u izazove i mogućnosti moderne analize podataka [2].

2.5 Metodologija rada

Projekt slijedi strukturiran pristup razvoja sustava za skladištenje podataka, koji se može podijeliti u pet glavnih faza:

1. **Eksploratorna analiza podataka** - detaljno istraživanje početnog skupa podataka radi razumijevanja strukture, kvalitete i potencijalnih izazova
2. **Dizajn relacijskog modela** - stvaranje normalizirane strukture podataka koja odražava realne poslovne entitete i njihove odnose
3. **Implementacija dimenzijskog modela** - razvoj star schema arhitekture optimizirane za analitičke potrebe

4. **ETL proces** - implementacija sustava za ekstrahiranje, transformaciju i učitavanje podataka koristeći Apache Spark
5. **OLAP analize** - demonstracija analitičkih mogućnosti kroz praktične scenarije kreiranja grafova i tablica u programu Tableau

Svaka faza dokumentirana je s objašnjenjima projektnih odluka, tehničkih izazova i načina njihova rješavanja, čineći projekt korisnim resursom za razumijevanje praktičnih aspekata razvoja sustava za skladištenje podataka.

3 Odabir i analiza skupa podataka

3.1 Izvor i kriteriji odabira podataka

Za potrebe ovog projekta odabran je javno dostupan skup podataka s Kaggle platforme pod nazivom "90,000+ Cars Data From 1970 to 2024" [5]. Ovaj dataset sadrži informacije o preko 97.000 automobila u razdoblju od 1970. do 2024. godine, što ga čini izvrsnim kandidatom za demonstraciju koncepata skladišta podataka i analitičkih sustava.

Kriteriji koji su vodili odabir ovog specifičnog skupa podataka uključuju:

- **Veličina i kompleksnost** - Dataset s preko 97.000 zapisa omogućuje demonstraciju tehnika rukovanja velikim količinama podataka karakterističnih za stvarne poslovne scenarije
- **Bogatstvo atributa** - Podatak sadrži 10 različitih atributa koji pokrivaju ključne aspekte automobilske tržišta: model, godinu proizvodnje, cijenu, vrstu mjenjača, kilometražu, tip goriva, porez, potrošnju, veličinu motora i proizvođača
- **Vremenski raspon** - Podaci se protežu kroz razdoblje od više desetljeća, što omogućuje analizu dugoročnih trendova i temporalnih promjena na tržištu
- **Praktična relevantnost** - Automobilska industrija predstavlja kompleksan sektor s jasnima hijerarhijama (proizvođač → model) i različitim kategorijama koje se prirodno mapiraju na dimenzijske modele podataka
- **Dostupnost i licenca** - Dataset je javno dostupan pod MIT licencom, što omogućuje slobodno korištenje u edukacijske svrhe

Odabrani dataset pruža solidan temelj za ilustraciju ključnih koncepata poslovne inteligencije i skladišta podataka, uključujući normalizaciju, denormalizaciju, ETL procese i OLAP analize.

3.2 Eksploratorna analiza podataka

Prije bilo kakve obrade ili transformacije podataka, provedena je detaljna eksploratorna analiza podataka (EDA) [6]. Ovaj korak je kritičan jer omogućuje razumijevanje strukture, kvalitete i karakteristika skupa podataka, što je neophodno za donošenje informiranih odluka o daljnjem pristupu [7].

3.2.1 Osnovna struktura podataka

Početni dataset sadrži sljedeće attribute:

Atribut	Tip podatka	Opis
model	string	Model automobila
year	integer	Godina proizvodnje
price	integer	Cijena u britanskim funtama
transmission	string	Tip mjenjača (Manual, Automatic, Semi-Auto)
mileage	integer	Kilometraža vozila
fuelType	string	Tip goriva (Petrol, Diesel, Hybrid, Electric)
tax	integer	Godišnji porez
mpg	float	Milje po galonu (potrošnja)
engineSize	float	Veličina motora u litrima
Manufacturer	string	Proizvođač automobila

Tablica 1: Struktura originalnog skupa podataka

Analiza je provedena koristeći Python biblioteke pandas i numpy, kako je prikazano u sljedećem kodu:

Listing 1: Segment skripte za analizu - osnovni pregled strukture podataka

```

1 import pandas as pd
2
3 # Ucitavanje podataka iz CSV datoteke
4 PATH = "../data/cars_data_original.csv"
5 data = pd.read_csv(PATH, delimiter=',')
6
7 # Ispis osnovnih informacija o skupu podataka
8 print("Dimenzije skupa podataka:", data.shape)
9 print("Tipovi podataka:")
10 print(data.dtypes)
11 print("Nedostaju vrijednosti:")
12 print(data.isna().sum())
13 print("Broj jedinstvenih vrijednosti po stupcima:")
14 print(data.nunique())
15 ...

```

3.2.2 Ključni uvidi iz eksploratorne analize

Analiza je otkrila nekoliko važnih karakteristika skupa podataka:

1. **Kvaliteta podataka** - Dataset ne sadrži nedostajuće vrijednosti (null values), što značajno pojednostavljuje faze čišćenja i pripreme podataka
2. **Problematične vrijednosti** - Identificirano je 268 automobila s veličinom motora od 0.0 litara, što upućuje na greške u podacima ili specifične kategorije vozila (električni automobili) koje zahtijevaju posebnu obradu
3. **Distribucija proizvođača** - Analiza je pokazala prisutnost 9 glavnih proizvođača: Ford, Volkswagen, BMW, Skoda, Toyota, Mercedes-Benz (označen kao "merc"), Vauxhall, Audi i Hyundai (označen kao "hyundi")

4. **Vremenski raspon** - Podaci pokrivaju godine od 1970. do 2024., s najvećom koncentracijom vozila iz 2010-ih godina
5. **Cijenski raspon** - Cijene se kreću od nekoliko stotina do preko 100.000 funti, što reflektira širokospan spektar od budget do luksuznih vozila

3.3 Priprema i obrada podataka

Na temelju uvida dobivenih iz eksploratorne analize, implementiran je proces pripreme podataka koji obuhvaća čišćenje, standardizaciju i proširenje originalnog skupa podataka. Ovaj proces je ključan za stvaranje kvalitetne osnove za kasnije faze razvoja skladišta podataka [8].

3.3.1 Čišćenje i standardizacija podataka

Prvi korak u pripremi podataka bio je proces čišćenja i standardizacije, implementiran u sljedećem kodu:

Listing 2: Čišćenje i standardizacija podataka

```
1 import pandas as pd
2
3 # Ucitavanje originalnog skupa podataka
4 df = pd.read_csv("data/cars_data_original.csv", delimiter=',',
5                 ',')
6
7 # Uklanjanje whitespace znakova iz naziva stupaca i podataka
8 df.columns = df.columns.str.strip()
9 df = df.apply(lambda x: x.str.strip() if x.dtype == "object"
10               else x)
11
12 # Pretvaranje cijena i kilometraže u numericke tipove
13 df['price'] = pd.to_numeric(df['price'])
14 df['mileage'] = pd.to_numeric(df['mileage'])
15
16 # Standardizacija naziva proizvođača
17 df = df.rename(columns={'Manufacturer': 'manufacturer'})
18 df['manufacturer'] = df['manufacturer'].str.lower()
19 df['manufacturer'] = df['manufacturer'].replace({
20     'bmw': 'BMW',
21     'merc': 'Mercedes-Benz',
22     'volkswagen': 'Volkswagen',
23     'toyota': 'Toyota',
24     'hyundai': 'Hyundai',
25     'vauxhall': 'Vauxhall',
26     'audi': 'Audi',
27     'skoda': 'Skoda',
28     'ford': 'Ford'
29 })
```

```

29 # Uklanjanje redaka s nedostajucim vrijednostima
30 df = df.dropna()

```

3.3.2 Proširenje skupa podataka hijerarhijskim atributima

Jedan od ključnih koraka u pripremi podataka za skladište bilo je proširenje originalnog skupa s dodatnim atributima koji omogućuju stvaranje bogatijih hijerarhija i boljih analitičkih mogućnosti. Ovaj proces je implementiran kroz dodavanje sljedećih dimenzija:

Listing 3: Proširenje skupa podataka novim dimenzijama

```

1 # Dodavanje desetljeća na temelju godine proizvodnje
2 df['decade'] = (df['year'] // 10 * 10).astype(str) + 's'
3
4 # Mapiranje proizvođača na zemlje i regije
5 manufacturer_mapping = {
6     'Hyundai': {'country': 'South Korea', 'region': 'Asia'},
7     'Volkswagen': {'country': 'Germany', 'region': 'Europe'},
8     },
9     'BMW': {'country': 'Germany', 'region': 'Europe'},
10    'Skoda': {'country': 'Czech Republic', 'region': 'Europe'},
11    },
12    'Ford': {'country': 'United States', 'region': 'North America'},
13    },
14    'Toyota': {'country': 'Japan', 'region': 'Asia'},
15    'Mercedes-Benz': {'country': 'Germany', 'region': 'Europe'},
16    },
17    'Vauxhall': {'country': 'United Kingdom', 'region': 'Europe'},
18    },
19    'Audi': {'country': 'Germany', 'region': 'Europe'}
20 }
21
22 # Dodavanje geografskih dimenzija
23 df['country'] = df['manufacturer'].map(
24     lambda mfr: manufacturer_mapping.get(mfr, {}).get('country', 'Unknown'))
25 df['region'] = df['manufacturer'].map(
26     lambda mfr: manufacturer_mapping.get(mfr, {}).get('region', 'Unknown'))
27
28 # Kategorizacija kilometraže
29 mileage_ranges = [
30     (0, 5000, 'Very Low'),
31     (5000, 20000, 'Low'),
32     (20000, 50000, 'Medium'),
33     (50000, 100000, 'High'),
34     (100000, 150000, 'Very High'),
35     (150000, float('inf'), 'Extreme')

```

```

31 ]
32
33 df['mileageCategory'] = 'Unknown'
34 for low, high, category in mileage_ranges:
35     mask = (df['mileage'] >= low) & (df['mileage'] < high)
36     df.loc[mask, 'mileageCategory'] = category
37
38 # Klasifikacija velicine motora
39 engine_size_classes = [
40     (0.1, 1.5, 'Small'),
41     (1.5, 2.5, 'Medium'),
42     (2.5, float('inf'), 'Large')
43 ]
44
45 df['engineSizeClass'] = 'Unknown'
46 for low, high, category in engine_size_classes:
47     mask = (df['engineSize'] >= low) & (df['engineSize'] <
48         high)
49     df.loc[mask, 'engineSizeClass'] = category
50
51 # Dodavanje starosti vozila i kategorizacije
52 current_year = 2025
53 df['age'] = current_year - df['year']
54
55 age_categories = [
56     (0, 3, 'New'),
57     (3, 7, 'Recent'),
58     (7, 12, 'Mature'),
59     (12, 20, 'Old'),
60     (20, float('inf'), 'Vintage')
61 ]
62
63 df['ageCategory'] = 'Unknown'
64 for low, high, category in age_categories:
65     mask = (df['age'] >= low) & (df['age'] < high)
66     df.loc[mask, 'ageCategory'] = category

```

3.3.3 Podjela skupa podataka za simulaciju različitih izvora

Za potrebe demonstracije ETL procesa koji može rukovati podacima iz različitih izvora, prošireni skup podataka podijeljen je na dva dijela u omjeru 80:20. Ovaj pristup omogućuje simulaciju realnog scenarija gdje skladište podataka prima informacije iz više nezavisnih sustava.

Listing 4: Podjela skupa podataka na dva dijela

```

1 import pandas as pd
2
3 # Ucitavanje prosirenog skupa podataka

```

```

4 df = pd.read_csv("processed/cars_data_EXPANDED.csv",
5                 delimiter=',')
6
7 # Nasumična podjela u omjeru 80:20
8 df20 = df.sample(frac=0.2, random_state=1)
9 df80 = df.drop(df20.index)
10
11 print(Velicina 80% skupa:", df80.shape)
12 print("Velicina 20% skupa:", df20.shape)
13
14 # Spremanje u zasebne datoteke
15 df80.to_csv("processed/cars_data_80.csv", index=False)
16 df20.to_csv("processed/cars_data_20.csv", index=False)

```

3.4 Rezultati pripreme podataka

Na završetku procesa pripreme podataka, od originalnog skupa s 10 atributa stvaren je prošireni dataset s 16 atributa koji uključuje:

- **Originalne attribute** - svih 10 osnovnih karakteristika vozila
- **Temporalne dimenzije** - decade, age, ageCategory
- **Geografske dimenzije** - country, region
- **Kategorizirane attribute** - mileageCategory, engineSizeClass

Ovakav pristup pripreme podataka osigurava da konačni skup sadrži sve potrebne elemente za stvaranje bogatog dimenzijskog modela koji može podržati kompleksne OLAP analize. Dodatne hijerarhije omogućuje implementaciju naprednih analitičkih scenarija poput drill-down operacija od regije prema specifičnim modelima automobila, ili roll-up agregacija od individualnih vozila prema tržišnim segmentima.

Pripravljeni podaci predstavljaju čvrstu osnovu za sljedeću fazu projekta - dizajn i implementaciju relacijskog modela podataka koji će služiti kao izvor za ETL proces prema dimenzijskom skladištu podataka.

4 Relacijski model podataka

Nakon uspješno provedene analize podataka, sljedeći korak u razvoju skladišta podataka predstavlja kreiranje relacijskog modela koji će osigurati strukturirano i normalizirano čuvanje podataka. Relacijski model omogućuje organizaciju podataka u tablice povezane preko stranih ključeva, što omogućava efikasno dohvaćanje i manipulaciju podataka [9].

4.1 Analiza entiteta i atributa

Temeljito razumijevanje strukture podataka o automobilima omogućilo je identifikaciju ključnih entiteta koji će činiti okosnicu relacijskog modela. Kroz analizu originalnog skupa podataka identificirani su sljedeći glavni entiteti:

Osnovni entiteti:

- **Automobil** - središnji entitet koji sadrži osnovne karakteristike vozila
- **Proizvođač** - entitet koji predstavlja tvrtke koje proizvode automobile
- **Model** - specifični model automobila određenog proizvođača
- **Zemlja** - zemlja podrijetla proizvođača
- **Regija** - geografska regija kojoj pripada zemlja

Klasifikacijski entiteti:

- **Tip mjenjača** - klasifikacija prema vrsti transmisije
- **Tip goriva** - kategorije goriva koje automobil koristi
- **Desetljeće** - vremenski period proizvodnje
- **Kategorija starosti** - klasifikacija prema godinama starosti
- **Kategorija kilometraže** - klasifikacija prema prijednim kilometrima
- **Klasa veličine motora** - kategorije prema volumenu motora

Ova kategorizacija omogućuje stvaranje normaliziranog modela koji minimizira redundanciju podataka i omogućuje efikasne upite.

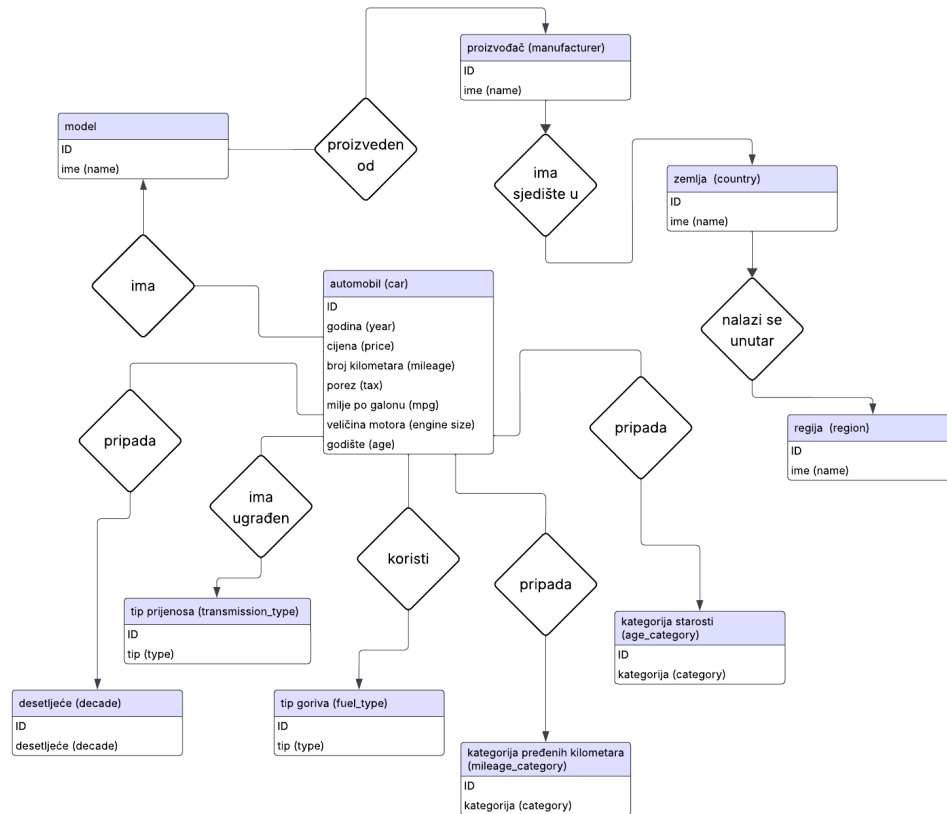
4.2 Konceptualni model podataka

Konceptualni model predstavlja visokorazinsku apstrakciju poslovnih zahtjeva bez ulaska u tehnička ograničenja. Za potrebe ovog projekta definiran je model koji odražava prirodne veze između entiteta u domeni trgovine automobilima.

Ključne veze u modelu uključuju:

- Svaki automobil pripada određenom modelu (1:N)
- Svaki model proizvodi točno jedan proizvođač (1:N)

- Svaki proizvođač dolazi iz jedne zemlje (1:N)
- Svaka zemlja pripada jednoj regiji (1:N)
- Automobil ima jednu kategoriju za svaki klasifikacijski atribut (1:N)

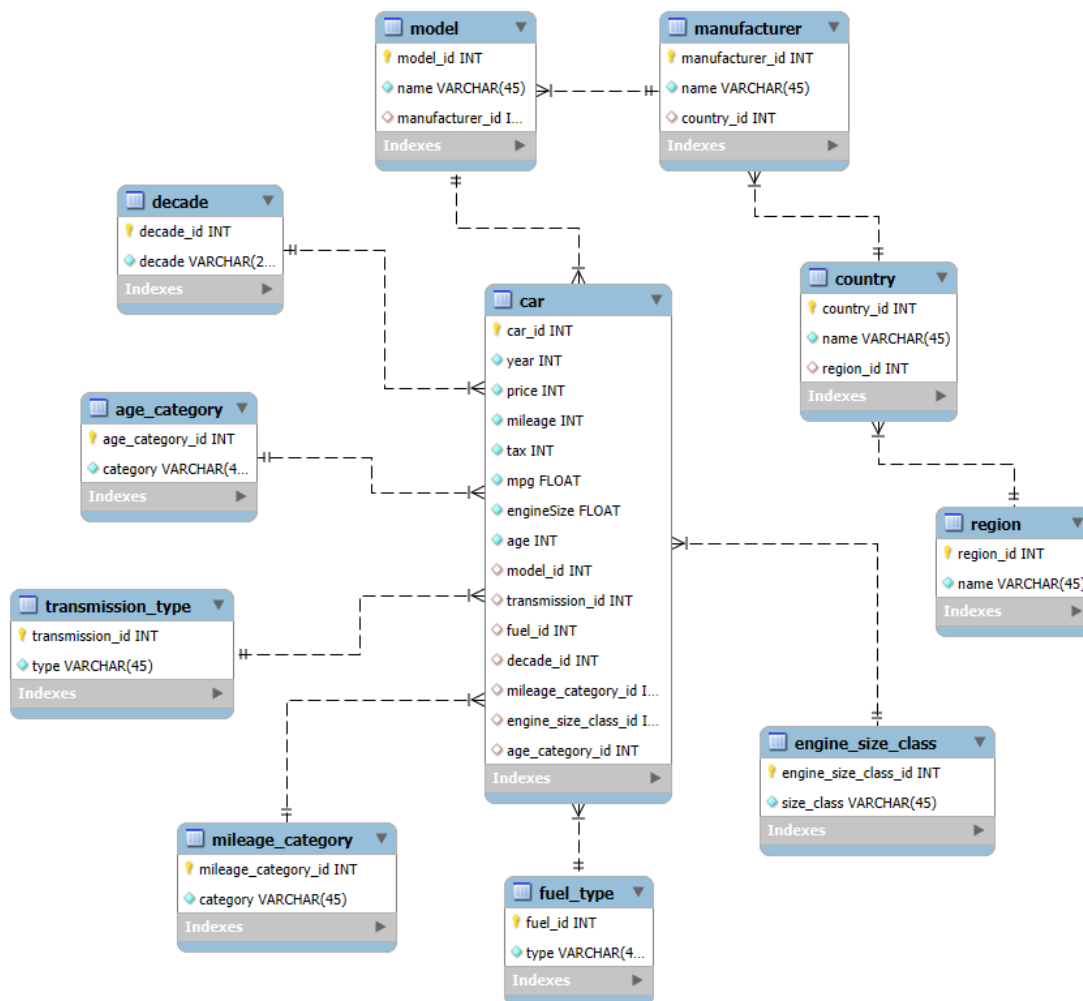


Slika 1: Konceptualni Entity-Relationship dijagram

ER dijagram prikazuje kompletan konceptualni model s entitetima, atributima i vezama između njih. Dijagram je kreiran koristeći standardnu notaciju koja jasno označava kardinalnosti i ograničenja. Na dijagramu je jasno vidljiva hijerarhijska struktura od regije do modela automobila, kao i klasifikacijski entiteti koji omogućuju analitičko izvještavanje. Svaki entitet sadrži primarni ključ i relevantne attribute koji podržavaju analitičke zahtjeve.

4.3 Implementacija relacijskog modela

Logički model podataka implementiran je koristeći MySQL bazu podataka, pri čemu je osobita pozornost posvećena normalnim formama i referencijalnim ograničenjima. Model slijedi treću normalnu formu (3NF) što osigurava minimizaciju redundancije podataka.



Slika 2: Implementirani relacijski model u MySQL bazi podataka

Implementacija uključuje:

- **Referencijalnu cjelovitost** - svi strani ključevi osiguravaju postojanje povezanih zapisa
- **Jedinstvene ograničenja** - sprječavanje dupliciranja entiteta
- **NOT NULL ograničenja** - osiguravanje kompletnosti kritičnih atributa
- **Auto-increment primarni ključevi** - efikasno upravljanje identifikatorima

4.4 Automatizacija kreiranja i popunjavanja baze

Za potrebe reproducibilnosti i održivosti projekta razvijene su Python skripte koje automatiziraju proces kreiranja i popunjavanja baze podataka. Glavna skripta koristi SQLAlchemy ORM za definiranje sheme baze.

Listing 5: Definiranje ORM modela za glavni entitet automobila

```

1 class Car(Base):
2     __tablename__ = 'car'
3     car_id = Column(Integer, primary_key=True, autoincrement
4         =True)
5     year = Column(Integer, nullable=False)
6     price = Column(Integer, nullable=False)
7     mileage = Column(Integer, nullable=False)
8     tax = Column(Integer, nullable=False)
9     mpg = Column(Float, nullable=False)
10    engineSize = Column(Float, nullable=False)
11    age = Column(Integer, nullable=False)
12
13    # Strani kljucevi
14    model_id = Column(Integer, ForeignKey('model.model_id'))
15    transmission_id = Column(Integer, ForeignKey('
16        transmission_type.transmission_id'))
17    fuel_id = Column(Integer, ForeignKey('fuel_type.fuel_id'
18        ))
19    # ... ostali strani kljucevi

```

Proces popunjavanja baze provodi se u kontroliranim fazama:

1. **Kreiranje osnovnih entiteta** - regije, zemlje, proizvođači
2. **Kreiranje klasifikacijskih tabela** - tipovi goriva, mjenjača, kategorije
3. **Kreiranje modela** - povezivanje s proizvođačima
4. **Umetanje automobila** - povezivanje sa svim referentnim entitetima

4.5 Validacija podataka i testiranje

Za osiguravanje ispravnosti procesa migracije podataka u zasebnoj skripti implementiran je sveobuhvatan sustav testiranja koji uspoređuje originalne CSV podatke s podacima u bazi.

Listing 6: SQL upit za rekonstrukciju originalnih podataka

```

1 query = """
2 SELECT mfg.name as 'manufacturer', mdl.name as 'model'
3 , c.year, c.price, c.mileage, c.tax, c.mpg, c.engineSize
4 , t.type as 'transmission', f.type as 'fuelType'
5 , d.decade, cnt.name as 'country', r.name as 'region'
6 , mc.category as 'mileageCategory'
7 , esc.size_class as 'engineSizeClass'
8 , c.age, ac.category as 'ageCategory'
9 FROM car c
10 JOIN model mdl ON c.model_id = mdl.model_id
11 JOIN manufacturer mfg ON mdl.manufacturer_id = mfg.
    manufacturer_id

```

```
12 -- ... ostali JOIN-ovi
13 ORDER BY c.car_id ASC
14 """
```

Test provjerava:

- **Integritet strukture** - postojanje svih stupaca u bazi
- **Kompletnost podataka** - broj zapisa u bazi odgovara CSV datoteci
- **Ispravnost sadržaja** - vrijednosti u bazi identične su originalnim podacima
- **Referencijalnu konzistentnost** - svi strani ključevi ispravno povezani

5 Dimenzijski model podataka

Nakon implementacije normaliziranog relacijskog modela, sljedeći korak u razvoju skladišta podataka predstavlja kreiranje dimenzijskog modela optimiziranog za analitičke upite i izvještavanje. Dimenzijski model, poznat i kao "star schema", omogućuje brže izvršavanje složenih analitičkih upita kroz denormalizirane strukture podataka [10].

5.1 Konceptualni pristup dimenzijskom modeliranju

Dimenzijski model predstavlja fundamentalno drugačiji pristup od tradicionalnog relacijskog modela. Umjesto fokusa na normalnu formu i eliminaciju redundancije, dimenzijski model optimizira strukturu za analitičke performance i razumljivost krajnjih korisnika [?].

Ključne karakteristike dimenzijskog pristupa uključuju:

Denormalizacija za performanse:

- Spojene tablice u široke dimenzije za smanjenje JOIN operacija
- Redundantni podaci prihvaćeni radi brzine upita
- Optimizacija za čitanje, a ne za pisanje podataka

Intuitivnost za analitičare:

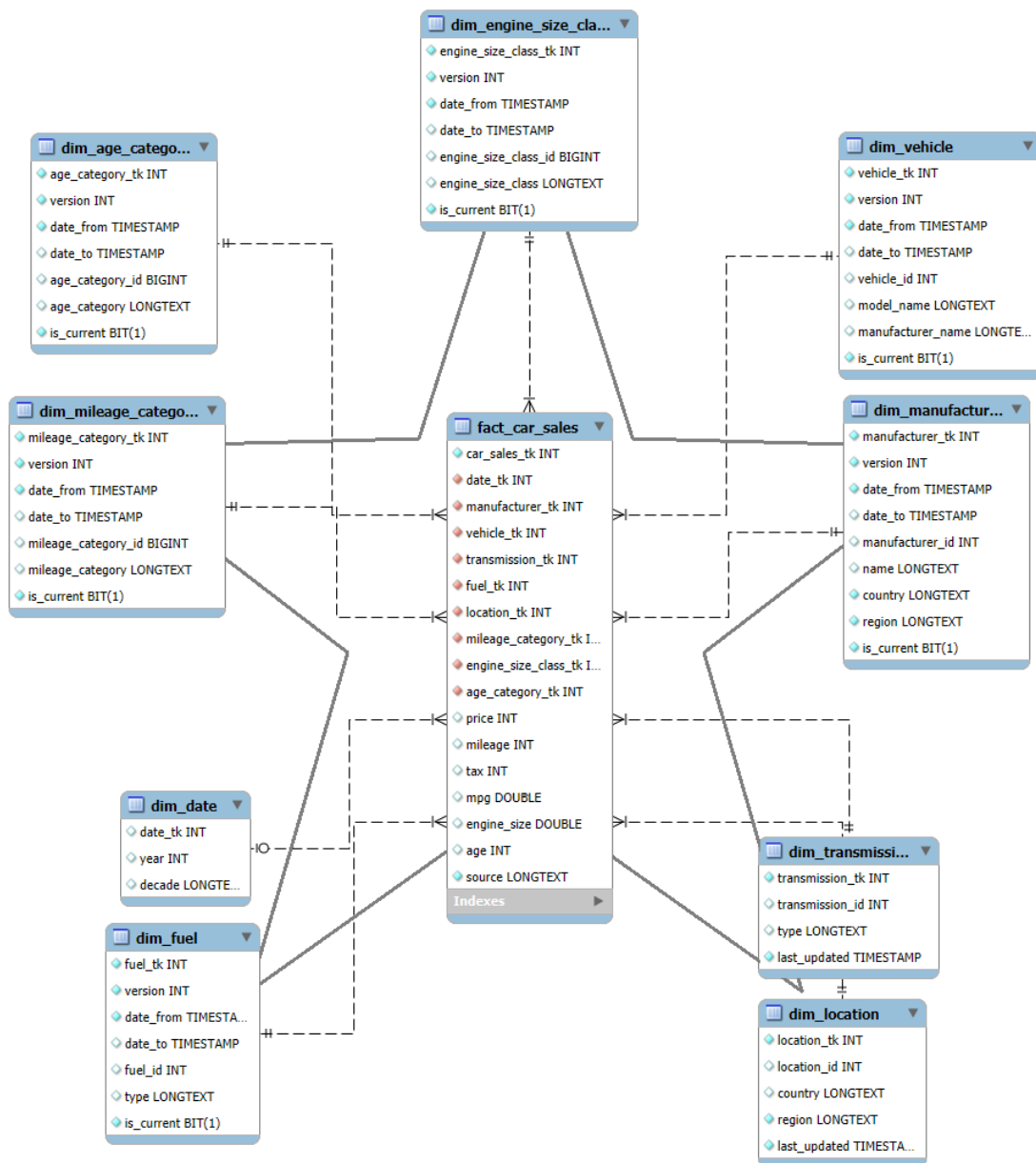
- Jasno razlikovanje između činjenica (mjera) i dimenzija (kontekst)
- Prirodno mapiranje na poslovne procese i izvještaje
- Jednostavnost za ad-hoc analize i OLAP operacije

Upravljanje promjenama kroz vrijeme:

- Implementacija Slowly Changing Dimensions (SCD) strategija
- Održavanje povijesnih podataka za trendove analize
- Fleksibilnost u dodavanju novih dimenzija i mjera

5.2 Arhitektura zvjezdastog modela

Za potrebe analize automobilskeg tržišta dizajnirana je star schema koja omogućuje dubinsku analizu prodajnih trendova, cijena i karakteristika vozila kroz različite dimenzije.



Slika 3: Star schema dimenzijskog modela za analizu automobilske tržišta

Model se sastoji od:

- **1 tablica činjenica** (Fact_Car_Sales) - centralna tablica s mjerama
- **9 dimenzijskih tablica** - kontekstualne informacije za analizu
- **Surrogate ključevi** - tehničke veze između tablica
- **SCD implementacija** - upravljanje promjenama kroz vrijeme

5.3 Tablica činjenica - Fact_Car_Sales

Središnja tablica činjenica predstavlja glavni repozitorij svih prodajnih činjenica vezanih uz automobile na tržištu. Tablica je optimizirana za agregacijske operacije i analitičke upite.

Listing 7: Definiranje tablice činjenica u SQLAlchemy ORM-u

```
1 class FactCarSales(Base):
2     __tablename__ = 'fact_car_sales'
3
4     car_sales_tk = Column(BigInteger, primary_key=True,
5                             autoincrement=True)
6
7     # Strani ključevi prema dimenzijama
8     date_tk = Column(Integer, ForeignKey('dim_date.date_tk'))
9     manufacturer_tk = Column(BigInteger, ForeignKey('dim_manufacturer.manufacturer_tk'))
10    vehicle_tk = Column(BigInteger, ForeignKey('dim_vehicle.vehicle_tk'))
11    transmission_tk = Column(BigInteger, ForeignKey('dim_transmission.transmission_tk'))
12    fuel_tk = Column(BigInteger, ForeignKey('dim_fuel.fuel_tk'))
13
14    # Mjere za analizu
15    price = Column(DECIMAL(10, 2), nullable=False) # Glavna mjera
16    mileage = Column(Integer, nullable=False)
17    tax = Column(Integer, nullable=False)
18    mpg = Column(DECIMAL(5, 2), nullable=False)
19    engine_size = Column(DECIMAL(3, 1), nullable=False)
20    age = Column(Integer, nullable=False)
```

Tablica sadrži sljedeće tipove atributa:

- **Surrogate ključ** - tehnički identifikator zapisa (car_sales_tk)
- **Strani ključevi** - veze s dimenzijskim tablicama
- **Mjere** - kvantitativni podaci za agregaciju i analizu
- **Audit polja** - praćenje kreiranja i ažuriranja podataka

5.4 Dimenzijske tablice i SCD strategije

Dimenzijske tablice pružaju kontekstualne informacije potrebne za interpretaciju činjenica. Ovisno o prirodi podataka, implementirane su različite SCD (Slowly Changing Dimensions) strategije.

SCD Type 0 - Statični podaci (Dim_Date):

- Podaci se nikad ne mijenjaju
- Jednostavna struktura bez povijesnih zapisa
- Koristi se za vremensku dimenziju

SCD Type 1 - Prepisivanje (Dim_Transmission, Dim_Location):

- Nova vrijednost zamjenjuje staru
- Nema čuvanja povijesti promjena
- Koristi se za podatke gdje povijest nije kritična

Listing 8: SCD Type 1 implementacija za tip mjenjača

```

1 class DimTransmission(Base):
2     __tablename__ = 'dim_transmission'
3
4     transmission_tk = Column(BigInteger, primary_key=True,
5                               autoincrement=True)
6     transmission_id = Column(Integer, nullable=False, index=
7                           True) # Business key
8     type = Column(String(20), nullable=False)
9     last_updated = Column(DateTime, default=datetime.now,
10                           onupdate=datetime.now)
11     created_date = Column(DateTime, default=datetime.now)

```

SCD Type 2 - Dodavanje novih zapisa (Dim_Manufacturer, Dim_Vehicle, ostali):

- Čuva se potpuna povijest promjena
- Svaka promjena kreira novi zapis
- Omogućuje analizu trendova kroz vrijeme

Listing 9: SCD Type 2 implementacija za proizvođače

```

1 class DimManufacturer(Base):
2     __tablename__ = 'dim_manufacturer'
3
4     manufacturer_tk = Column(BigInteger, primary_key=True,
5                               autoincrement=True)
6     version = Column(Integer, nullable=False, default=1)
7     date_from = Column(DateTime, nullable=False, default=
8                     datetime.now)
9     date_to = Column(DateTime, nullable=True)
10    manufacturer_id = Column(Integer, nullable=False, index=
11                          True) # Business key
12    name = Column(String(100), nullable=False)
13    country = Column(String(50), nullable=False)

```



```

11     region = Column(String(50), nullable=False)
12     is_current = Column(Boolean, nullable=False, default=
        True)

```

5.5 Prednosti zvjezdastog modela

Implementacija star schema modela donosi značajne prednosti za analitičke procese:

Performanse upita:

- Smanjenje složenosti JOIN operacija
- Optimizacija za agregacijske funkcije
- Predictable execution planovi za OLAP upite

Skalabilnost:

- Linearan rast s volumenom podataka
- Mogućnost horizontalnog particioniranja
- Kompatibilnost s distribuiranim analitičkim sustavima

Jednostavnost korištenja:

- Intuitivna struktura za poslovne analitičare
- Izravno mapiranje na OLAP kočke
- Kompatibilnost s BI alatima (Tableau, Power BI, itd.)

5.6 Implementacijski detalji

Kreiranje dimenzijskog modela automatizirano je kroz Python skriptu koja osigurava konzistentnost strukture i referencijalnih veza.

Listing 10: Kreiranje star schema strukture

```

1  # Kreiranje baze podataka za data warehouse
2  temp_engine = create_engine('mysql+pymysql://root:
    root@localhost', echo=False)
3  with temp_engine.connect() as conn:
4      conn.execute(text("CREATE DATABASE IF NOT EXISTS cars_dw
        CHARACTER SET utf8mb4"))
5      conn.commit()
6
7  # Kreiranje svih tablica u dimenzijskom modelu
8  Base.metadata.drop_all(engine)  # Uklanjanje postojećih
    tablica
9  Base.metadata.create_all(engine)  # Kreiranje novih tablica
10
11 print("Star Schema tables created successfully!")

```

Model je spreman za ETL proces koji će popuniti dimenzije i tablicu činjenica podacima iz relacijske baze, pri čemu će se implementirati potrebne SCD strategije za upravljanje promjenama kroz vrijeme.

6 ETL proces

ETL (Extract, Transform, Load) proces predstavlja srce svakog skladišta podataka, omogućujući transformaciju heterogenih izvornih podataka u konzistentan, analitički optimiziran format. Za potrebe ovog projekta implementiran je sveobuhvatan ETL pipeline koji kombinira podatke iz relacijske baze i CSV datoteka, transformira ih u dimenzijski model i učitava u skladište podataka.

6.1 Arhitektura ETL pipelinea

ETL proces projektiran je prema modernim načelima scalable data processing korištenjem Apache Spark okruženja koje omogućuje distribuirano procesiranje velikih količina podataka [11].

Ključne komponente arhitekture uključuju:

Tehnološki stack:

- **Apache Spark** - distribuirano procesiranje podataka
- **PySpark** - Python API za Spark s DataFrame optimizacijama
- **JDBC konektor** - povezivanje s MySQL bazama podataka
- **SQLAlchemy ORM** - objektno-relacionalno mapiranje za schema management

Modularni dizajn:

- Razdvojene Extract, Transform i Load faze
- Nezavisan kod za svaku dimenziju i tablicu činjenica
- Mogućnost pokretanja pojedinačnih komponenti za testiranje
- Centralizirano upravljanje Spark sesijama

Listing 11: Inicijalizacija Spark okruženja za ETL pipeline

```
1 def get_spark_session(app_name="ETL_App"):  
2     project_root = os.path.dirname(os.path.dirname(os.path.  
3         abspath(__file__)))  
4     mysql_jar_path = os.path.join(project_root, "Connectors"  
5         , "mysql-connector-j-9.2.0.jar")  
6  
7     return SparkSession.builder \  
8         .appName(app_name) \  
9         .config("spark.jars", mysql_jar_path) \  
10        .config("spark.driver.host", "127.0.0.1") \  
11        .config("spark.sql.adaptive.enabled", "false") \  
12        .config("spark.serializer", "org.apache.spark.  
13            serializer.KryoSerializer") \  
14        .getOrCreate()
```

6.2 Extract faza - izvlačenje podataka

Extract faza odgovorna je za dohvaćanje podataka iz heterogenih izvora i njihovo ujedinjavanje u Spark DataFrame strukture pogodne za daljnju obradu.

Izvlačenje iz MySQL baze podataka:

Implementiran je generički sustav za dohvaćanje svih tablica iz relacijske baze koristeći JDBC konektor koji omogućuje optimiziranu komunikaciju između Spark-a i MySQL-a.

Listing 12: Izvlačenje podataka iz MySQL relacijske baze

```
1 def extract_table(table_name, database="cars"):
2     spark = get_spark_session("ETL_Extract_MySQL")
3
4     jdbc_url = f"jdbc:mysql://127.0.0.1:3306/{database}?
5         useSSL=false&allowPublicKeyRetrieval=true"
6     connection_properties = {
7         "user": "root",
8         "password": "root",
9         "driver": "com.mysql.cj.jdbc.Driver"
10    }
11
12    try:
13        df = spark.read.jdbc(url=jdbc_url, table=table_name,
14            properties=connection_properties)
15        return df
16    except Exception as e:
17        print(f"Error extracting table {table_name}: {e}")
18        return spark.createDataFrame([], "struct<id:int>")
```

Izvlačenje iz CSV datoteka:

Za fleksibilnost i potporu dodatnim izvorima podataka, implementirana je mogućnost učitavanja podataka iz CSV datoteka koje mogu sadržavati nove zapise ili dopunske informacije.

Listing 13: Izvlačenje podataka iz CSV datoteka

```
1 def extract_from_csv(file_path):
2     spark = get_spark_session("ETL_Extract - CSV")
3     df = spark.read.option("header", True).option("
4         inferSchema", True).csv(file_path)
5     return df
```

6.3 Transform faza - transformacija podataka

Transform faza predstavlja najkompleksniji dio ETL procesa gdje se izvlačeni podaci transformiraju u dimenzijski model optimiziran za analitičke operacije.

6.3.1 Transformacija dimenzijskih tablica

Svaka dimenzijska tablica implementira odgovarajuću SCD strategiju prema prirodi podataka i poslovnim zahtjevima:

SCD Type 0 - Statična vremenska dimenzija:

Listing 14: Transformacija vremenske dimenzije

```
1 def transform_date_dim(car_df, csv_cars_df=None):
2     # Izvlacenje jedinstvenih godina iz baze
3     mysql_df = (
4         car_df
5         .select(col("year"))
6         .dropDuplicates()
7         .withColumn("decade", concat(((col("year").cast("int")
8             ) / 10).cast("int") * 10).cast("string"), lit("s
9             )))
10         .select("year", "decade")
11     )
12
13     # Dodavanje surrogate ključa
14     final_df = mysql_df.withColumn("date_tk", col("year"))
15     return final_df
```

SCD Type 1 - Prepisivanje za jednostavne entitete:

Koristi se za podatke poput tipova mjenjača gdje povijest promjena nije kritična.

Listing 15: SCD Type 1 implementacija za tip mjenjača

```
1 def transform_transmission_dim(transmission_type_df,
2     csv_cars_df=None):
3     merged_df = (
4         transmission_type_df
5         .select(
6             col("transmission_id"),
7             trim(col("type")).alias("type")
8         )
9         .withColumn("type", initcap(trim(col("type"))))
10        .dropDuplicates()
11    )
12
13    # Dodavanje audit polja
14    merged_df = merged_df.withColumn("last_updated",
15        current_timestamp())
16
17    # Surrogate key
18    window = Window.orderBy("type")
19    merged_df = merged_df.withColumn("transmission_tk",
20        row_number().over(window))
21
22    return merged_df
```

SCD Type 2 - Praćenje povijesti za ključne entitete:

Implementiran je za proizvođače, vozila i tipove goriva gdje je održavanje povijesti promjena kritično za analize trendova.

Listing 16: SCD Type 2 implementacija za proizvođače

```
1 def transform_manufacturer_dim(manufacturer_df, country_df,
2   region_df, csv_cars_df=None):
3     # Spajanje s geografskim informacijama
4     merged_df = (
5         manufacturer_df.alias("m")
6         .join(country_df.alias("c"), col("m.country_id") ==
7             col("c.country_id"), "left")
8         .join(region_df.alias("r"), col("c.region_id") ==
9             col("r.region_id"), "left")
10        .select(
11            col("m.manufacturer_id").alias("manufacturer_id"),
12            trim(col("m.name")).alias("name"),
13            trim(col("c.name")).alias("country"),
14            trim(col("r.name")).alias("region")
15        )
16    )
17
18    # SCD Type 2 polja
19    merged_df = merged_df.withColumn("version", lit(1)) \
20        .withColumn("date_from",
21            current_timestamp()) \
22        .withColumn("date_to", lit(None).
23            cast("timestamp")) \
24        .withColumn("is_current", lit(True))
25
26    return merged_df
```

6.3.2 Transformacija tablice činjenica

Tablica činjenica predstavlja najveći izazov zbog potrebe za spajanjem podataka iz svih dimenzija i osiguravanjem referencijalnih veza.

Listing 17: Kreiranje tablice činjenica

```
1 def transform_car_sales_fact(raw_data, manufacturer_dim,
2   vehicle_dim, transmission_dim,
3   fuel_dim, location_dim, date_dim,
4   mileage_category_dim,
5   engine_size_class_dim,
6   age_category_dim):
7
8     # Priprema činjenica iz baze podataka
```

```

6     fact_df = prepare_fact_from_database(raw_data["car"],
7                                         raw_data)
8
9     # Dodavanje podataka iz CSV-a ako postoje
10    if "csv_cars" in raw_data and raw_data["csv_cars"] is
11        not None:
12        csv_fact_df = prepare_fact_from_csv(raw_data["
13            csv_cars"])
14        fact_df = fact_df.unionByName(csv_fact_df)
15
16    # Dimension lookups za Foreign Key-eve
17    fact_with_dims = perform_dimension_lookups(
18        fact_df, manufacturer_dim, vehicle_dim,
19        transmission_dim,
20        fuel_dim, location_dim, date_dim,
21        mileage_category_dim,
22        engine_size_class_dim, age_category_dim
23    )
24
25    return fact_with_dims

```

Ključni aspekti transformacije:

- **Data cleansing** - uklanjanje neispravnih i nedosljednih podataka
- **Standardizacija** - ujedinjavanje formata i kodiranja
- **Surrogate key generiranje** - kreiranje tehničkih identifikatora
- **Dimension lookup** - mapiranje business ključeva na surrogate ključeve
- **Data enrichment** - dodavanje kategorijskih klasifikacija

6.4 Load faza - učitavanje u skladište podataka

Load faza odgovorna je za efikasno i sigurno učitavanje transformiranih podataka u ciljno skladište podataka, pri čemu se poštuju referencijalna ograničenja i optimiziraju performanse.

Strategija učitavanja:

Implementiran je pristup koji poštuje hijerarhiju foreign key veza između tablica, učitavajući najprije dimenzije, a zatim tablice činjenica.

Listing 18: Kontrolirano učitavanje s poštovanjem foreign key veza

```

1 def write_all_tables_to_mysql(load_ready_dict):
2     # Prvo uklanjanje tablica činjenica za izbjegavanje FK
3     konflikata
4     drop_fact_tables_first()
5
6     # Redoslijed učitavanja: dimenzije prvo

```

```

6     dimension_tables = ['dim_manufacturer', 'dim_vehicle', '
7         dim_transmission',
8         'dim_fuel', 'dim_location', 'dim_date',
9         'dim_mileage_category', '
10         dim_engine_size_class', '
11         dim_age_category']
12     fact_tables = ['fact_car_sales']
13
14     # Učitavanje dimenzija
15     for table_name in dimension_tables:
16         if table_name in load_ready_dict:
17             write_spark_df_to_mysql(load_ready_dict[
18                 table_name], table_name)
19
20     # Zatim učitavanje činjenica
21     for table_name in fact_tables:
22         if table_name in load_ready_dict:
23             write_spark_df_to_mysql(load_ready_dict[
24                 table_name], table_name)

```

JDBC optimizacije:

Učitavanje koristi JDBC batch operacije za optimalne performanse, s mogućnosti fallback-a na CSV export u slučaju tehničkih problema.

Listing 19: Optimizirano JDBC učitavanje s fallback strategijom

```

1 def write_spark_df_to_mysql(spark_df: DataFrame, table_name:
2     str, mode: str = "overwrite"):
3     jdbc_url = "jdbc:mysql://127.0.0.1:3306/cars_dw?useSSL=
4         false&allowPublicKeyRetrieval=true"
5     connection_properties = {
6         "user": "root",
7         "password": "root",
8         "driver": "com.mysql.cj.jdbc.Driver"
9     }
10
11     try:
12         spark_df.write.jdbc(
13             url=jdbc_url,
14             table=table_name,
15             mode=mode,
16             properties=connection_properties
17         )
18         print(f"Uspjesno ucitano {spark_df.count()} zapisa u
19             tablicu {table_name}")
20
21     except Exception as e:
22         print(f"Greska pri ucitavanju: {e}")
23         # Fallback na CSV export

```



```

21     output_path = f"4_etl/output/{table_name}.csv"
22     spark_df.coalesce(1).write.mode(mode).option("header", "true").csv(output_path)

```

6.5 Orkestracija i monitoring

Kompletan ETL pipeline orchestriran je kroz glavnu skriptu koja koordinira sve faze i omogućuje praćenje napretka.

Listing 20: Glavna ETL orkestracija

```

1  def main():
2      spark = get_spark_session()
3      spark.sparkContext.setLogLevel("ERROR")
4
5      # Extract faza
6      print("Pocinje izvlacenje podataka...")
7      mysql_df = extract_all_tables()
8      csv_df = {"csv_cars": extract_from_csv("2
          _relational_model/processed/cars_data_20.csv")}
9      merged_df = {**mysql_df, **csv_df}
10     print("Izvlacenje podataka završeno")
11
12     # Transform faza
13     print("Pocinje transformacija podataka...")
14     load_ready_dict = run_transformations(merged_df)
15     print("Transformacija podataka završena")
16
17     # Load faza
18     print("Pocinje učitavanje podataka...")
19     write_all_tables_to_mysql(load_ready_dict)
20     print("ETL proces uspješno završen")

```

6.6 Prednosti implementiranog ETL rješenja

Razvijeni ETL proces donosi značajne prednosti za upravljanje skladištem podataka:

Skalabilnost i performanse:

- Apache Spark omogućuje distribuirano procesiranje velikih volumena
- Lazy evaluation optimizira izvršavanje kompleksnih transformacija
- In-memory computing ubrzava iterativne operacije

Fleksibilnost i proširivost:

- Modularni dizajn omogućuje lako dodavanje novih izvora podataka
- Podrška za različite SCD strategije prema poslovnim zahtjevima

- Mogućnost pokretanja pojedinačnih komponenti za testiranje i debugging

Pouzdanost i održivost:

- Fallback strategije za slučajeve tehničkih problema
- Detaljno logiranje za praćenje i dijagnostiku
- Transakcijska sigurnost kroz kontrolirano učitavanje

Ovakav pristup osigurava da skladište podataka bude ažurno, konzistentno i spremno za složene analitičke operacije potrebne za poslovne analize i izvještavanje [12].

7 OLAP analiza

7.1 Definiranje prikaza podataka

7.2 Vizualizacija podataka u Tableau

7.2.1 Graf 1 TODO

8 Zaključak

Literatura

- [1] N. Silva, J. Barros, M. Y. Santos, C. Costa, and P. Cortez. Advancing Logistics 4.0 with the Implementation of a Big Data Warehouse: A Demonstration Case for the Automotive Industry. *Electronics*, 10(18):2221, 2021. [Na internetu]. Dostupno: <https://www.mdpi.com/2079-9292/10/18/2221> [pristupano 29. kolovoza 2025.].
- [2] G. Garani, A. Chernov, and I. Savvas. A Data Warehouse Approach for Business Intelligence. In *2019 IEEE 28th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*, pages 70–75, 2019. [Na internetu]. Dostupno: <https://ieeexplore.ieee.org/document/8795395> [pristupano 29. kolovoza 2025.].
- [3] P. Nima. Data Warehousing and Business Intelligence Project on Car Insights in the United Kingdom. ResearchGate, 2018. [Na internetu]. Dostupno: <https://www.researchgate.net/publication/330837638> [pristupano 29. kolovoza 2025.].
- [4] B. Leka, D. Leka, and B. Baraku. Driving Operational Excellence: Business Intelligence in the Car Parts Industry. *WSEAS Transactions on Business and Economics*, 22:356–367, 2025. [Na internetu]. Dostupno: <https://www.wseas.com/journals/bae/2025/a505118-356.pdf> [pristupano 29. kolovoza 2025.].
- [5] Meruvu Likith. 90,000+ Cars Data From 1970 to 2024. Kaggle, 2022. [Na internetu]. Dostupno: <https://www.kaggle.com/datasets/meruvulikith/90000-cars-data-from-1970-to-2024> [pristupano 29. kolovoza 2025.].
- [6] M. Komorowski, D. C. Marshall, J. D. Saliccioli, and Y. Crutain. Exploratory Data Analysis. In *Secondary Analysis of Electronic Health Records*, pages 185–203. Springer, 2016. [Na internetu]. Dostupno: https://link.springer.com/chapter/10.1007/978-3-319-43742-2_15 [pristupano 29. kolovoza 2025.].
- [7] N. Ekbote and P. Dhanshetti. Techniques of Exploratory Data Analysis. *Madhya Pradesh Journal of Social Sciences*, 28(2):45–58, 2023. [Na internetu]. Dostupno: <https://www.researchgate.net/publication/374674185> [pristupano 29. kolovoza 2025.].
- [8] Z. Wu and Z. Wu. Exploration, Visualization, and Preprocessing of High-Dimensional Data. In *Statistical Methods in Molecular Biology*, pages 163–177. Springer, 2009. [Na internetu]. Dostupno: https://link.springer.com/protocol/10.1007/978-1-60761-580-4_8 [pristupano 29. kolovoza 2025.].
- [9] R. Elmasri and S. B. Navathe. *Fundamentals of Database Systems*. Pearson, 7th edition, 2017. [Na internetu]. Dostupno: <https://www.pearson.com/us/higher-education/program/>

[Elmasri-Fundamentals-of-Database-Systems-7th-Edition/PGM189052.html](#) [pristupano 29. kolovoza 2025.].

- [10] R. Kimball and M. Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3rd edition, 2013. [Na internetu]. Dostupno: <https://www.wiley.com/en-us/The+Data+Warehouse+Toolkit%3A+The+Definitive+Guide+to+Dimensional+Modeling%2C+3rd+Edition-p-9781118530801> [pristupano 29. kolovoza 2025.].
- [11] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica. Apache Spark: A Unified Engine for Big Data Processing. In *Communications of the ACM*, volume 59, pages 56–65, 2016. [Na internetu]. Dostupno: <https://dl.acm.org/doi/10.1145/2934664> [pristupano 29. kolovoza 2025.].
- [12] S. Chaudhuri, U. Dayal, and V. Narasayya. An Overview of Business Intelligence Technology. *Communications of the ACM*, 54(8):88–98, 2011. [Na internetu]. Dostupno: <https://dl.acm.org/doi/10.1145/1978542.1978562> [pristupano 29. kolovoza 2025.].

Popis slika

1	Konceptualni Entity-Relationship dijagram	12
2	Implementirani relacijski model u MySQL bazi podataka	13
3	Star schema dimenzijskog modela za analizu automobilskeg tržišta .	17

Popis tablica

1	Struktura originalnog skupa podataka	6
---	--	---